DeployGuard

Self-Hosted Runtime Migration & Implementation Architecture

Single-Server FYP Design Report

Purpose: define exactly what changes when DeployGuard moves from per-application AWS ECS infrastructure to a pooled, self-hosted container runtime on one Linux VPS - without reopening the certified repository-analysis and BuildPlan pipeline.

Prepared: 24 August 2026

Scope: architecture, replacements, implementation phases, monitoring, self-healing, autoscaling, persistence, security, testing and FYP positioning

Status: proposed target architecture - NOT yet implemented or certified

Executive Decision

DeployGuard can realistically evolve from an AWS-specific execution plane into a low-cost, self-hosted PaaS runtime while preserving the strongest part of the existing system: deterministic repository analysis, exact ENV ownership, BuildPlan authority, immutable image construction and executable preflight. The proposed first target is one Linux VPS running Docker Engine in single-node Swarm mode, Traefik for routing/TLS, and the existing Prometheus/Grafana observability stack adapted to local runtime metrics.

> **Recommended architecture: keep the certified pipeline above BuildPlan/preflight unchanged; replace the AWS execution plane below that boundary with a provider-agnostic RuntimeNode adapter.**

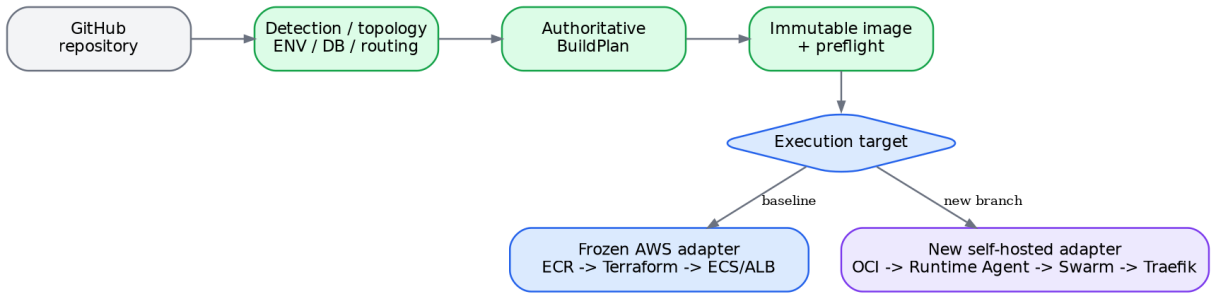


Figure 1. Preserve the certified control pipeline and add a self-hosted execution adapter.

Why this direction is defensible

- The FYP scope is JavaScript/Python web deployment; the provider is an implementation choice, not the core research contribution. The proposal originally selected AWS ECS Fargate, but its stated motivation includes reducing deployment complexity and cost for SMEs [F1].
- The latest scope audit certifies the bounded JavaScript/Python deployment contract with zero critical blind guesses, zero critical late failures and zero supported-scope defects; therefore the repository compiler should be frozen rather than redesigned [C1].
- Docker Swarm provides declarative services, desired-state reconciliation, replica scaling, resource limits, rolling updates and rollback, which are sufficient orchestration primitives for a single-node FYP runtime [D1][D2].
- Academic container-orchestration literature identifies fault tolerance, availability, resource utilization, scheduling and request throughput as central platform concerns; pooled container execution is therefore a valid systems problem, not merely 'running Docker' [A2][A3].
- Single-node self-healing can recover container/task failures, but it cannot recover physical VPS failure. The report keeps that boundary explicit.

What this report does not claim

- It does not claim the self-hosted runtime is already implemented or certified.
- It does not claim one VPS provides host-level high availability.
- It does not promise a fixed number of applications per server; capacity must be measured.
- It does not enable autoscaling for arbitrary stateful applications by guess.
- It does not require Kubernetes/K3s for the first release.
- It does not reintroduce unsupported workers, application Redis, arbitrary Compose-first or arbitrary Dockerfile-only topologies.

Contents

1. Certified Baseline and Change Boundary
2. Cost and Product Differentiation
3. Target Runtime Architecture
4. Technology Choice: Swarm vs Compose vs K3s
5. AWS-to-Self-Hosted Replacement Matrix
6. Runtime Node and Runtime Agent Design
7. Image Build and Registry Design
8. Networking, Routing, DNS and TLS
9. Deployment, Candidate Verification, Promotion and Rollback
10. Resource Isolation and Admission Control
11. Monitoring, Logging and Alerting
12. Self-Healing Design
13. Autoscaling Design for One Server
14. Managed Databases, Persistence and Backup
15. Security Model
16. Backend Modules, Data Model and APIs
17. UI / Dashboard Design
18. State and Failure Semantics
19. Implementation Plan
20. Test and Certification Plan
21. Cost/Capacity Evaluation Method
22. Single-Node Limitations and Honest Claims
23. FYP Document and Panel Positioning
24. Risks, Mitigations and Rollback Plan
25. Future Multi-Node Roadmap
26. Final Recommended MVP

References

1. Certified Baseline and Change Boundary

The most important architecture rule is that the self-hosted migration must not reopen repository semantics. The current FYP audit reports the JavaScript web scope, Python web scope, bounded frontend+backend scope and managed database scope as certified, with zero critical blind guesses, zero late failures, zero regressions and zero supported-scope defects [C1].

Layer	Current status	Migration decision
Repository / stack detection	Certified bounded scope	KEEP
ENV discovery, ownership and secret classification	Certified	KEEP
Topology and relationship evidence	Certified	KEEP
BuildPlan / runtime contract	Certified	KEEP
Docker generation / custom supported mode	Certified	KEEP
Executable preflight	Certified	KEEP
GitHub Actions build execution	Working active path	KEEP
AWS ECR / Terraform / ECS / ALB application execution	Certified baseline	REPLACE
CloudWatch-specific runtime integration	AWS-specific	REPLACE
Prometheus / Grafana presentation	Already implemented in final product	KEEP
Self-hosted RuntimeNode / Agent	Not present	NEW
Swarm service lifecycle and capacity admission	Not present	NEW

> **Migration invariant: RuntimeSpec must be a mechanical projection of the frozen BuildPlan. The self-hosted adapter may transform representation, but must not re-detect ENV, ports, health or routing.**

2. Cost and Product Differentiation

The proposed business advantage is not 'we do not use a third party'. A VPS provider is still third-party infrastructure. The difference is economic and architectural: DeployGuard purchases a shared pool of compute and safely multiplexes several certified applications onto it, instead of provisioning a large set of cloud resources around each small application.

Model	Cost behavior	DeployGuard value
Current AWS execution	Per-application cloud resources create a high baseline even for low-traffic apps	Automatic infrastructure compilation, but expensive execution substrate
Self-hosted pooled runtime	One node cost is shared across admitted workloads	Repository-aware admission + container orchestration + pooled cost
Commercial PaaS	Provider defines runtime rules and pricing	DeployGuard differentiates through deterministic repository contract and portable runtime

Illustrative current market evidence: Hostinger listed a 4-vCPU / 16-GB / 200-GB KVM VPS at USD 12.99/month introductory and USD 28.99/month renewal on 24 August 2026 [H1]. The team has observed roughly USD 55/month for its current AWS application setup. These numbers must not be turned into a percentage-savings claim until capacity tests determine how many representative applications the node can safely host.

```
Effective infrastructure cost per application =
  (VPS + backup/object storage + domain/other runtime costs)
  / number of concurrently admitted applications
  at a defined reliability/performance test profile
```

> **Do not market 'X% cheaper' before measuring safe capacity. Make cost-per-certified-workload an experiment in the FYP.**

3. Target Runtime Architecture

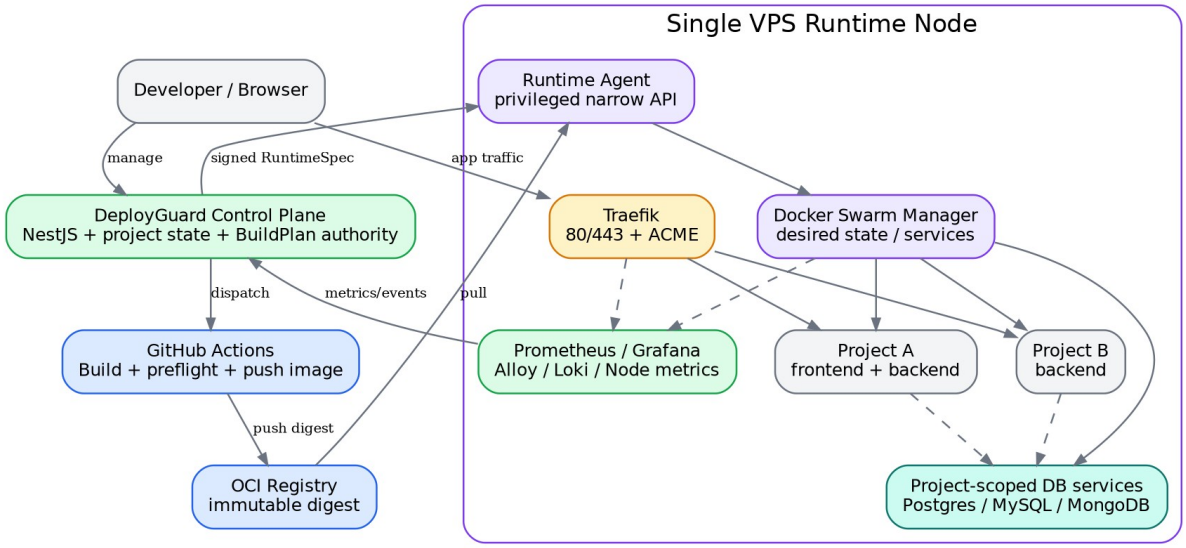


Figure 2. Recommended one-VPS architecture.

3.1 Logical planes

Plane	Responsibilities	Failure effect
Control plane	Project state, BuildPlan authority, deployment commands, UI, AI troubleshooting	Management operations pause if unavailable
Build plane	GitHub Actions builds and preflights immutable images	New deployments pause; running apps continue
Runtime plane	Docker Swarm services, networks, volumes, routing and self-heal	Customer runtime impact
Observability plane	Prometheus, logs, dashboards, alerts and runtime events	Reduced visibility; should not stop healthy traffic

3.2 One physical server, multiple logical isolation zones

Because only one VPS exists initially, DeployGuard must protect its own control/monitoring capacity from customer workloads. The node should reserve measured CPU, memory and disk headroom for the OS, Docker daemon, Traefik, DeployGuard services and monitoring. Workload admission consumes only the remaining pool. The reserve is a platform policy derived from measurement; it must not be a hidden per-repository guess.

4. Technology Choice: Swarm vs Compose vs K3s

Option	Strengths	Weaknesses for this FYP	Decision
Plain Docker / Compose	Simplest start; familiar	No strong desired-state service model, replica orchestration or first-class Swarm secrets	Reject as main runtime
Docker Swarm	Declarative services, desired-state reconciliation, scaling, overlay networks, secrets, resource limits, rolling update/rollback [D1][D2]	Smaller ecosystem than Kubernetes; single node has no host HA	RECOMMENDED
K3s / Kubernetes	Powerful autoscaling ecosystem and multi-node future	More control-plane concepts and operational surface than required for one FYP server	Future option

Docker documents Swarm services as a declarative desired-state model: the service specification includes image, replicas, ports, restart/update behavior and placement/resource constraints, and the manager reconciles actual state toward that desired state [D1][D2]. That gives DeployGuard real orchestration behavior without forcing the team to build a Kubernetes-sized platform.

5. AWS-to-Self-Hosted Replacement Matrix

AWS concept	Replacement	Design note
ECS Service	Docker Swarm Service	Desired replicas, restart policy, update/rollback policy
ECS Task	Swarm task/container	Immutable image instance
ALB	Traefik	Host/path routing and TLS
ALB Target Group	Traefik service/router	Backend selection and load balancing
ECR	OCI registry (GHCR recommended MVP)	Immutable digest distribution
Secrets Manager	DeployGuard secret store + Swarm secrets	Service-scoped runtime secret delivery
CloudWatch metrics	Prometheus + node/container/Traefik metrics	Host, workload and HTTP telemetry
CloudWatch logs	Docker stdout/stderr -> Alloy -> Loki	Centralized searchable logs
EFS managed DB persistence	Project-scoped local named volumes	Survives container replacement; not host loss
AWS scaling policy	DeployGuard autoscaling controller + Swarm replicas	Single-node workload scaling only
AWS candidate health	Runtime Agent + internal probe utility	Exact BuildPlan TCP/HTTP/relationship checks
Terraform application resources	RuntimeSpec -> Runtime Agent -> Swarm API	Shared node is persistent; app deploy is service lifecycle

6. Runtime Node and Runtime Agent Design

6.1 RuntimeNode abstraction

Do not create a Hostinger-specific deployment service. Model a generic Linux RuntimeNode so the same execution adapter can later target Hostinger, Namecheap, Hetzner, an on-premises lab machine or a cloud VM.

```
RuntimeNode {
  id
  providerLabel        // informational, not deployment logic
  hostname / address
  status               // ONLINE | DEGRADED | OFFLINE
  totalCpuCores
  totalMemoryBytes
  totalDiskBytes
  reservedSystemCpu
  reservedSystemMemory
  allocatableCpu
  allocatableMemory
  lastHeartbeatAt
  dockerEngineVersion
  swarmNodeId
}
```

6.2 Why use a Runtime Agent

The NestJS backend should not expose or pass the Docker socket to customer workloads. A small root-owned Runtime Agent should be the only DeployGuard component allowed to invoke privileged Docker/Swarm operations. On the first

one-server version, the backend can communicate with the agent through a Unix domain socket, which avoids opening a privileged TCP management endpoint.

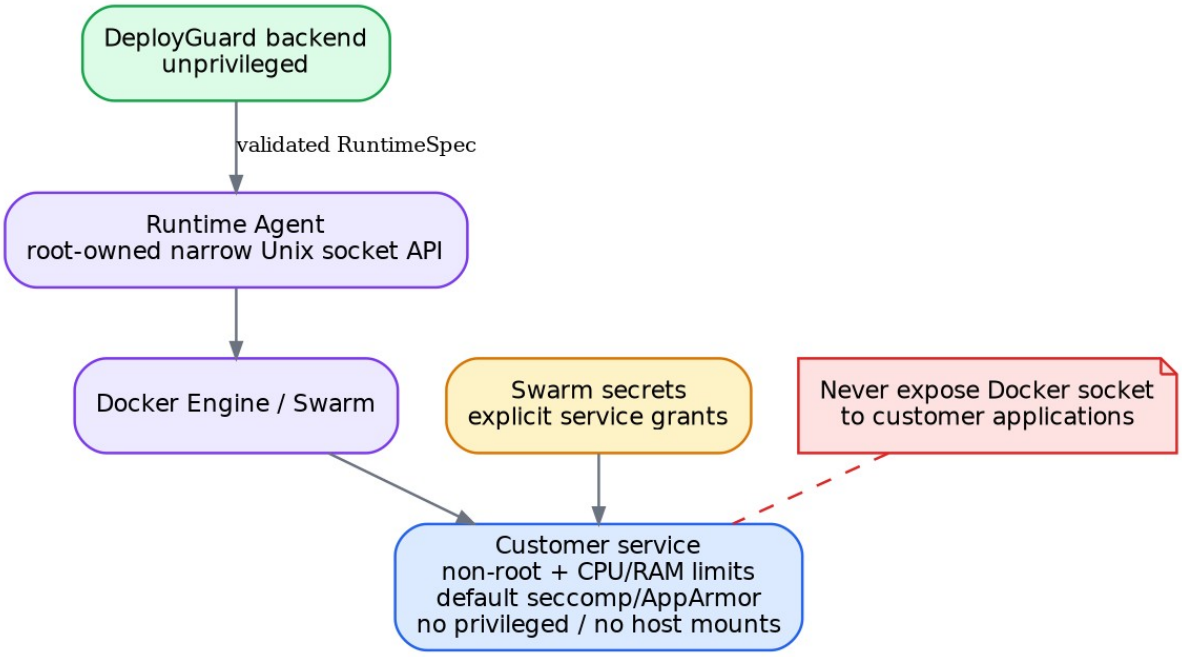


Figure 3. Narrow privileged boundary around Docker/Swarm.

- Run agent as a systemd service owned by root.
- Expose a Unix socket such as /run/deployguard-agent.sock to a dedicated deployguard group.
- Accept only a strict RuntimeSpec schema - never arbitrary shell strings.
- Use argument-based process execution; reject unsupported mounts, capabilities, devices and network modes.
- Return structured operation IDs and evidence rather than raw unrestricted Docker access.
- Future multi-node version can replace the local socket with mutually authenticated TLS.

6.3 Runtime Agent API surface

Operation	Purpose	Must validate
POST /deployments/candidate	Create release-specific candidate services	RuntimeSpec hash, image digest, capacity, networks, secret ownership
POST /deployments/{id}/verify	Run exact health/relationship probes	Only BuildPlan-proven probe modes/paths
POST /deployments/{id}/promote	Attach/switch public route	Candidate verification evidence
POST /deployments/{id}/rollback	Restore prior release spec	Prior immutable release exists
DELETE /projects/{id}/runtime	Destroy app services/networks/secrets/volumes per policy	Project ownership
POST /services/{id}/scale	Change desired replicas	Stateless eligibility, min/max policy, capacity admission
GET /node/capacity	Return measured runtime capacity	Host metrics available
GET /events	Read structured runtime events	Project/operation scoping

7. Image Build and Registry Design

Keep builds off the VPS. GitHub Actions already builds the immutable image and executes the application preflight. Building arbitrary repositories on the shared runtime would allow CPU, RAM and disk-heavy build work to compete with LIVE applications.

- GitHub Actions: checkout -> BuildPlan validation -> immutable image build -> executable preflight -> push by digest.

- Registry interface: standard OCI image reference; GHCR is a practical MVP default because GitHub is already required by the FYP.
- Runtime never deploys a mutable 'latest' tag. Store and deploy the digest.
- The RuntimeSpec includes the image digest and the BuildPlan generation/hash that produced it.
- Registry credentials are platform credentials and must never be injected into customer containers.

```
RuntimeSpec.image = {
  repository: "ghcr.io/<owner>/<project>/<component>",
  digest: "sha256:...",
  sourceCommit: "...",
  buildPlanGeneration: "...",
  buildPlanHash: "..."
}
```

8. Networking, Routing, DNS and TLS

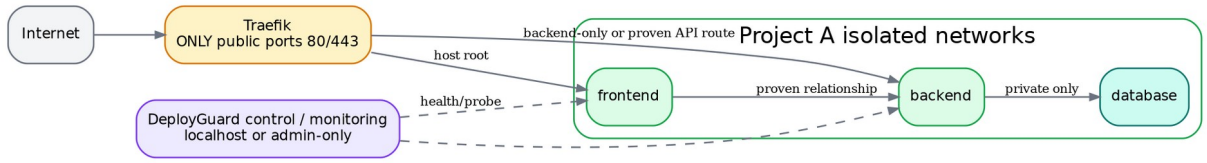


Figure 4. Public edge plus project-private network isolation.

8.1 Network rules

- Only Traefik publishes host ports 80 and 443.
- Each project receives a private overlay network with a deterministic project-owned name.
- Managed databases attach only to the private project network; never to the public edge network.
- A backend-only web project may attach its backend service to the edge network for Traefik routing.
- For frontend+backend, the self-hosted adapter should compile the already-proven BuildPlan relationship into Traefik rules or an equivalent runtime config without rediscovering preserve/strip semantics.
- No customer service may use host network mode.

8.2 Traefik mapping

Traefik's Swarm provider reads labels from Swarm services and requires the backend port to be specified explicitly in Swarm mode [T1]. This aligns well with DeployGuard because BuildPlan already owns the proven application port.

```
BuildPlan:
  publicHost = <generated project host>
  backend.port = 8000
  relationship.prefix = /api/v1
  relationship.proxyBehavior = PRESERVE | STRIP | UNKNOWN

Runtime compilation:
  Host(project.apps.example.com) -> public service
  PathPrefix(/api/v1) -> backend service when relationship is proven
  STRIP -> add strip-prefix middleware
  PRESERVE -> no strip middleware
  UNKNOWN -> deployment remains BLOCKED
```

> **Do not use Traefik to invent /api, /health, ports or strip/preserve behavior. It only materializes BuildPlan facts.**

8.3 Default DNS/TLS strategy

- Create one wildcard DNS A/AAAA record such as *.apps.deployguard.example pointing to the VPS.
- Use per-project hostnames such as <project>-<shortid>.apps.deployguard.example.

- Traefik obtains per-host certificates using ACME HTTP-01 through ports 80/443.
- Use the ACME staging endpoint during repeated development tests to avoid certificate-rate issues.
- Custom user domains are future scope unless time permits.

9. Deployment, Candidate Verification, Promotion and Rollback

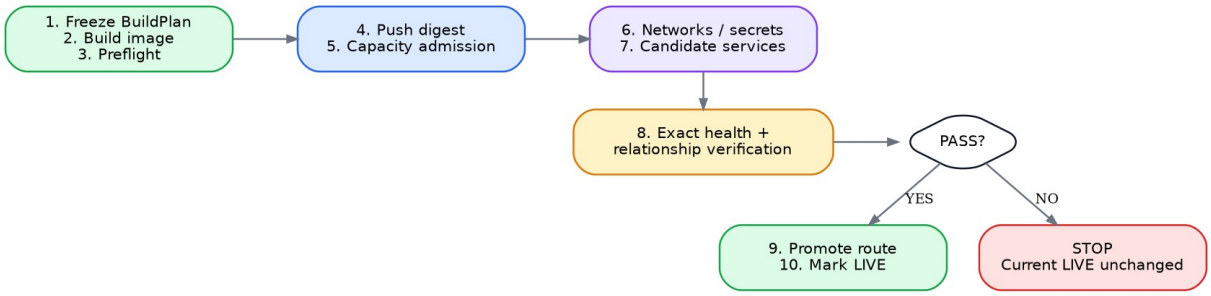


Figure 5. Proposed self-hosted deployment lifecycle.

9.1 Preserve the candidate model

The current candidate-before-promotion approach should be preserved because it is one of DeployGuard's strongest reliability properties. Creating a self-hosted runtime is not a reason to downgrade to 'update LIVE and hope'.

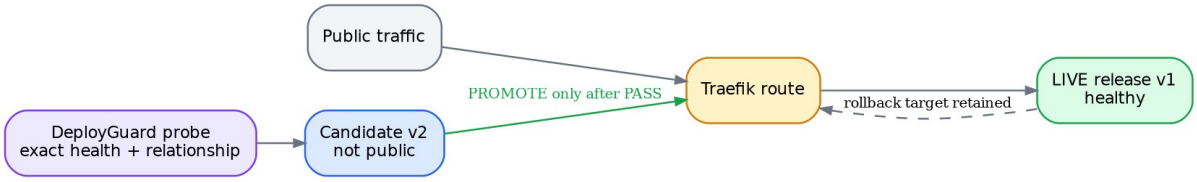


Figure 6. LIVE remains authoritative until the candidate passes.

9.2 Candidate implementation options

Option	Description	Recommendation
Release-specific candidate services	Create candidate service names and private verification route/network, then promote public routing	Preferred for preserving current semantics
Direct Swarm rolling update	Update LIVE service image and use Swarm automatic rollback	Useful fallback / simpler path but changes current candidate guarantee

For the FYP branch, prefer release-specific candidate services. A small platform-owned probe container can join the candidate network and perform the exact TCP/HTTP checks defined by the BuildPlan. Relationship verification should run only when a concrete verification method is proven; a router prefix must never be converted into a GET endpoint.

9.3 Promotion

1. Candidate services reach desired state.
2. Exact startup/health checks pass.
3. Exact frontend/backend relationship verification passes when provable.
4. Traefik labels/routes are changed atomically or through a bounded service update.
5. Current release becomes previous-release metadata.
6. Candidate becomes LIVE.
7. If promotion/update fails within the monitored release window, restore the prior service specification or route.

Swarm supports rolling update behavior and service rollback, including automatic rollback on failed updates [D2]. DeployGuard should still remain the product-level authority for whether a release is promoted.

10. Resource Isolation and Admission Control

Pooled infrastructure is only cheaper if one application cannot destroy the host. Docker states that containers have no CPU or memory constraints by default, and an out-of-memory condition can destabilize the whole system; explicit limits are therefore mandatory [D3].

```
Node allocatable capacity =
  measured host capacity
  - system/control-plane reserve
  - safety headroom

Admit deployment only if:
  sum(active service reservations)
  + candidate reservations
  <= allocatable capacity
```

Policy	MVP behavior	Reason
Memory hard limit	Required for every service	Prevent host-wide OOM
Memory reservation	Required / measured profile	Admission accounting
CPU limit	Required	Prevent noisy-neighbor CPU starvation
Replica min/max	Explicit platform policy	Bound resource growth
No privileged containers	Always reject	Host protection
No Docker socket mounts	Always reject	Docker daemon is privileged
No arbitrary host bind mounts	Always reject	Tenant filesystem isolation
Project-private network	Required	Reduce cross-project reachability
Disk free-space safety floor	Required host admission check	Protect registry/DB/log operations

> **Resource defaults are platform-owned policy, not repository facts. They should be tuned by controlled load/capacity tests and documented.**

11. Monitoring, Logging and Alerting

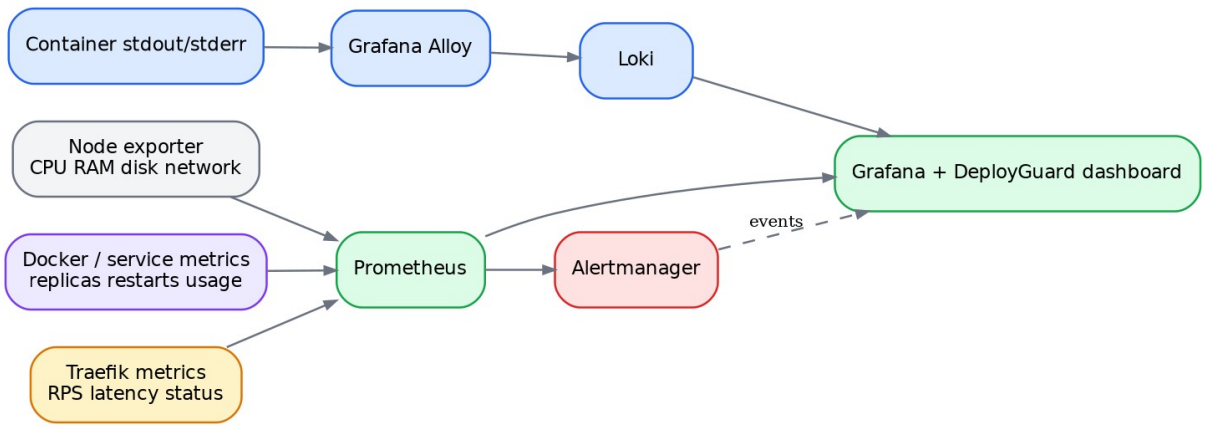


Figure 7. Proposed one-node observability pipeline.

The final product already includes Prometheus/Grafana concepts, so the self-hosted migration should replace AWS-specific telemetry sources rather than replace the dashboard stack. Node Exporter exposes host hardware/kernel metrics [P1]. Google SRE recommends focusing user-facing monitoring on latency, traffic, errors and saturation [S1].

11.1 Minimum metrics

Scope	Metrics	Use
Node	CPU, memory, filesystem free, network, load, Docker daemon health	Capacity / saturation / admission
Service	desired/current replicas, restart count, task state, CPU/memory usage	Self-heal and runtime health
HTTP edge	requests/sec, status classes, p50/p95/p99 latency	Traffic, error and latency signals
Database	service up, connections, storage usage, backup age	Persistence health
DeployGuard	deployment duration, failed stage, self-heal count, scale events	Platform evidence

11.2 Logging

Customer applications continue to write stdout/stderr. Grafana Alloy can discover Docker containers and collect their logs; Loki stores/query them and Grafana presents them [G1]. Keep log ingestion non-blocking: if Loki fails, healthy application traffic should continue.

11.3 Alerts

Prometheus Alertmanager can group, deduplicate, route, silence and inhibit alerts [P2]. Start with a small actionable set rather than alerting on every metric.

- Host memory or disk critically low.
- No healthy replica for a LIVE service.
- Repeated restart loop.
- Sustained HTTP 5xx rate above policy.
- p95 latency above a measured service SLO during load tests.
- Database unavailable.
- Backup overdue/failed.
- Node agent heartbeat lost.

12. Self-Healing Design

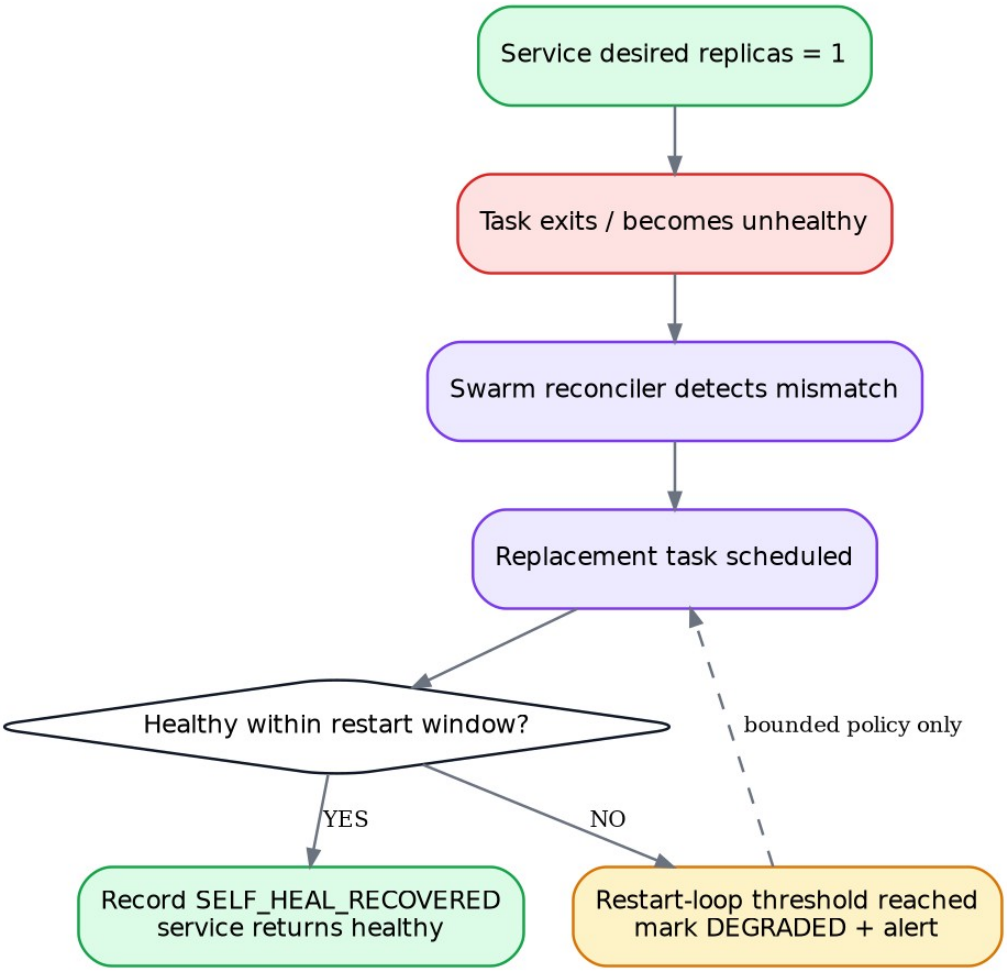


Figure 8. Single-node task self-healing loop.

Swarm desired-state reconciliation gives DeployGuard legitimate service-level self-healing: when a task exits, the manager can create a replacement to restore the desired replica count [D1][D2]. Research on self-adaptive microservices reports strong use of reactive infrastructure-level self-healing approaches [A4].

12.1 Honest self-healing claims

Failure	Can one VPS self-heal?	Behavior
Application process crash	YES	Swarm replaces failed task
Container health failure	YES	Restart/replacement according to service policy
Bad candidate deployment	YES	Do not promote / rollback update
Repeated restart loop	PARTIAL	Detect, stop endless churn, mark DEGRADED, alert
Docker daemon restart	PARTIAL	Services recover when daemon/host returns
VPS kernel/hardware/provider failure	NO	No second node exists; requires provider recovery or future multi-node runtime

12.2 Runtime event evidence

```
RuntimeEvent examples:
```

```
TASK_FAILED
SELF_HEAL_STARTED
SELF_HEAL_RECOVERED
RESTART_LOOP_DETECTED
RELEASE_ROLLBACK_STARTED
RELEASE_ROLLBACK_COMPLETED
NODE_SATURATED
NODE_HEARTBEAT_LOST
```

13. Autoscaling Design for One Server

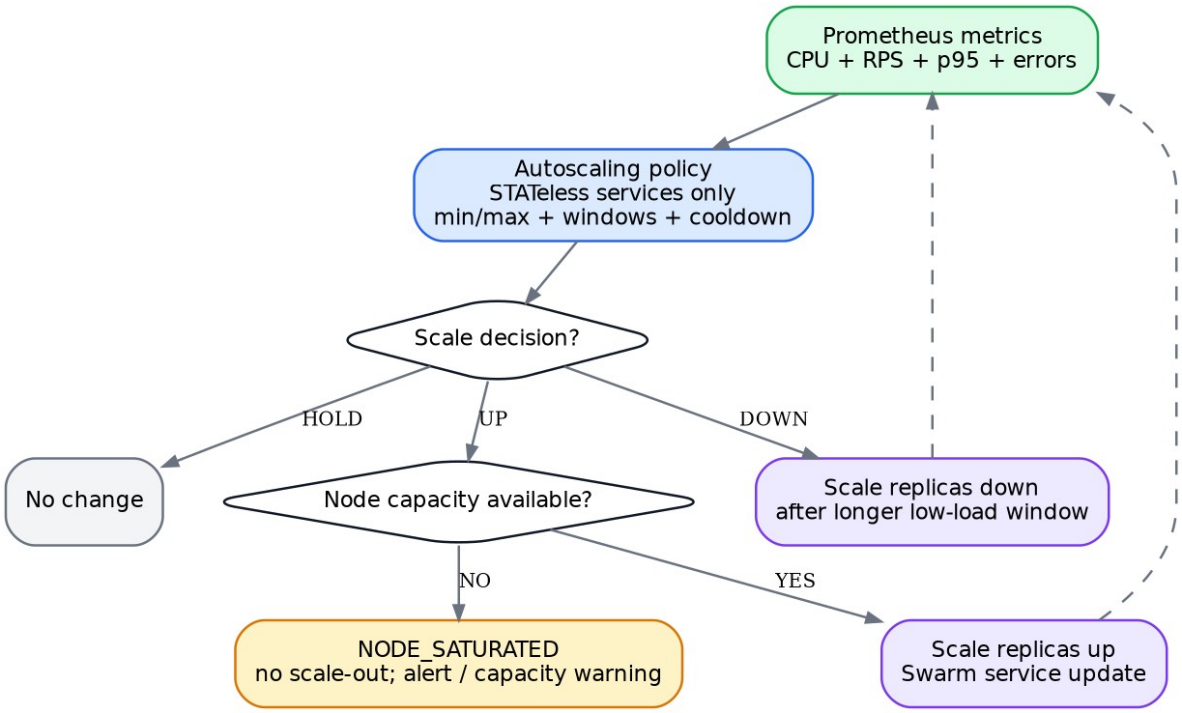


Figure 9. Capacity-aware workload autoscaling loop.

Autoscaling on one server means changing the number of application replicas inside fixed physical capacity. It can improve concurrency and resilience to a single process failure, but it does not create additional CPU/RAM. True infrastructure autoscaling requires another node.

13.1 Eligibility

> **Do not automatically horizontally scale an arbitrary backend unless DeployGuard can prove the service is safe to replicate. In-memory sessions, local writable state or singleton jobs can make naive scaling incorrect.**

- Autoscaling should initially be enabled only for certified stateless test fixtures and explicitly opted-in services.
- Managed database services remain scale=1 in the one-node MVP.
- Frontend static services are natural scale candidates.
- If statelessness is UNKNOWN, min=max=1.

13.2 Controller algorithm

A simple reactive controller is sufficient for the FYP: query Prometheus on a fixed interval, evaluate sustained windows, apply hysteresis/cooldown, check node capacity, then update Swarm desired replicas. The thresholds themselves are platform policy and should be tuned by the load-testing study rather than copied from convention.

```
every control_interval:
  metrics = read(service CPU, RPS, p95 latency, error rate)

  if service.autoscaling not eligible:
      continue

  desired = current

  if sustained_high_load and current < max:
      if capacity_service.canReserve(one_more_replica):
          desired = current + 1
      else:
          emit NODE_SATURATED

  else if sustained_low_load and cooldown_elapsed and current > min:
      desired = current - 1

  if desired != current:
      swarm.scale(service, desired)
      emit SCALE_EVENT
```

13.3 How to test without real traffic

Synthetic load is the correct test mechanism. k6 supports ramping virtual users and pass/fail thresholds for metrics such as HTTP error rate and response latency [K1][K2]. Run k6 from a developer machine or GitHub-hosted runner - not from the same VPS - so the load generator does not consume the capacity being measured.

8. Deploy a stateless Express/FastAPI certification fixture at 1 replica.
9. Ramp virtual users or arrival rate in stages.
10. Record RPS, p95 latency, error rate, CPU and memory.
11. Confirm the autoscaler scales 1 -> 2 only after the sustained policy condition.
12. Confirm Traefik balances across both replicas.
13. Stop load and confirm cooldown/hysteresis before 2 -> 1.
14. Repeat until the node saturation boundary is found.
15. Export the k6 HTML report and Prometheus/Grafana evidence for the FYP appendix.

14. Managed Databases, Persistence and Backup

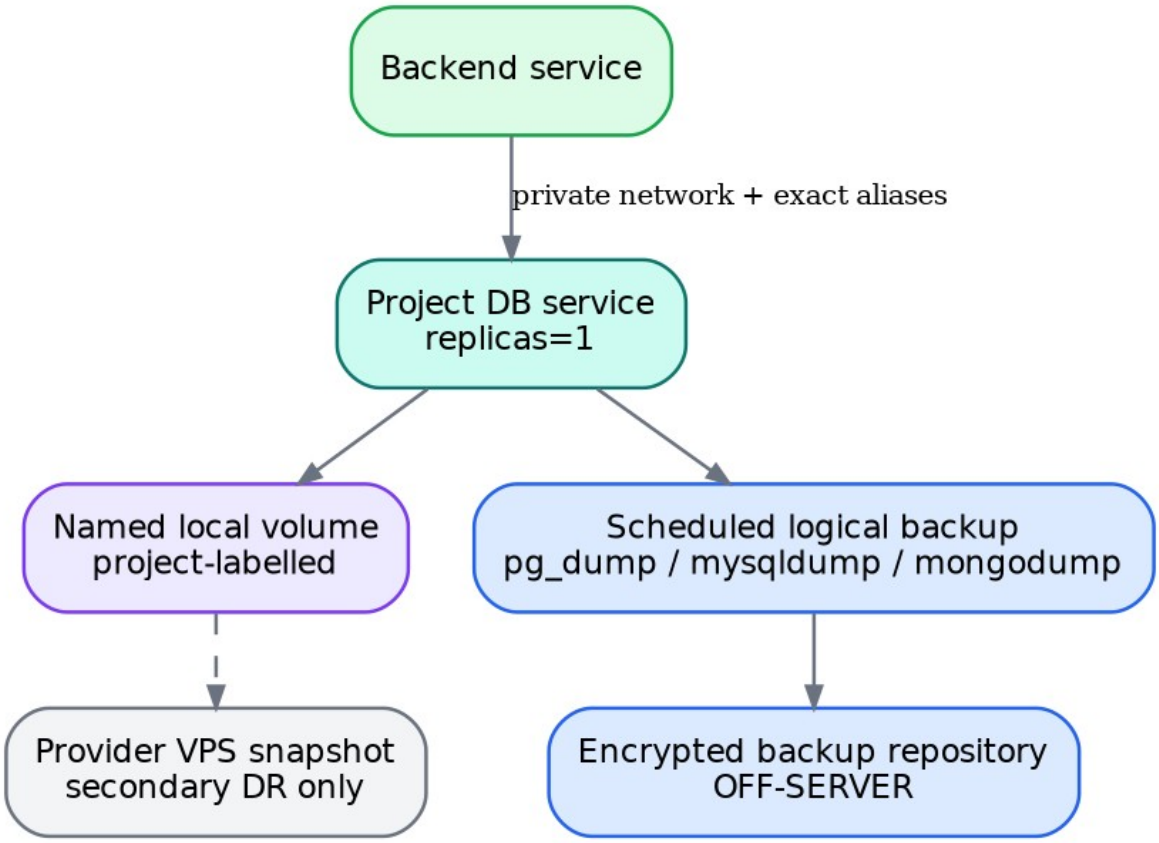


Figure 10. Project-scoped database persistence and off-server backup.

Preserve the existing managed-database semantic model. PostgreSQL, MySQL and MongoDB remain project-scoped services with exact application-facing ENV aliases. The runtime mechanism changes from ECS/EFS to a private Swarm service plus a project-labeled named volume.

Property	One-node design
Database replicas	1 only
Network	Project-private; never public
Credentials	DeployGuard-owned secret -> Swarm secret / controlled runtime materialization
Storage	Named local volume with deterministic project ownership labels
Container restart	Data persists
Server disk loss	Data does NOT survive without off-server backup
Backup	Scheduled logical dump + encrypted off-server repository
Provider snapshot	Secondary disaster-recovery layer, not application-consistent primary backup

Docker's own secret documentation demonstrates stateful services at scale 1 with private networks and persistent volumes, and warns that database scaling is not achieved merely by increasing replica count [D4].

> **Do not claim database high availability on one node. Claim container-restart persistence plus backup-based disaster recovery.**

15. Security Model

Self-hosting shifts more security responsibility to DeployGuard. The default Docker seccomp profile is designed as a moderately protective allowlist and Docker recommends not disabling it [D5]. Swarm secrets are encrypted in transit and at rest in the swarm Raft log and are granted only to authorized services [D4].

Control	Required MVP decision
Customer runtime user	Non-root unless repository custom image proves a safe explicit user
Privileged	Forbidden
Docker socket	Forbidden in customer services
Host PID/IPC/network	Forbidden
Host bind mounts	Forbidden except platform-owned explicit cases
Linux capabilities	Drop unnecessary capabilities; do not add arbitrary capabilities
Seccomp	Keep Docker default; never use unconfined as a general workaround
AppArmor/LSM	Keep host/default protection where supported
CPU/memory	Mandatory hard limits
Networks	Per-project isolation; DB private only
Secrets	Service-scoped; never frontend/private cross-contamination
Management API	Local Unix socket first; no unauthenticated remote Docker API
Firewall	Public 80/443 only; SSH restricted; monitoring/admin not public by default

16. Backend Modules, Data Model and APIs

Exact code paths must be chosen after inspecting the experimental branch, but the following responsibilities should be introduced without scattering Docker/Swarm commands across existing project services.

Proposed responsibility	Purpose
DeploymentTarget / RuntimeAdapter interface	Compile the existing BuildPlan into a target-specific RuntimeSpec
SelfHostedSwarmAdapter	Self-hosted execution orchestration only
RuntimeNodeService	Node registration, heartbeat, status and capacity
RuntimeAgentClient	Strict communication with privileged agent
RuntimeDeploymentService	Candidate/create/promote/rollback/destroy lifecycle
RuntimeCapacityService	Reservations, admission and saturation decisions
RuntimeHealthService	Exact TCP/HTTP/relationship verification
RuntimeEventService	Task/restart/self-heal/scale event persistence
RuntimeAutoscalingService	Optional stateless service control loop
RuntimeBackupService	Managed DB backup state/evidence

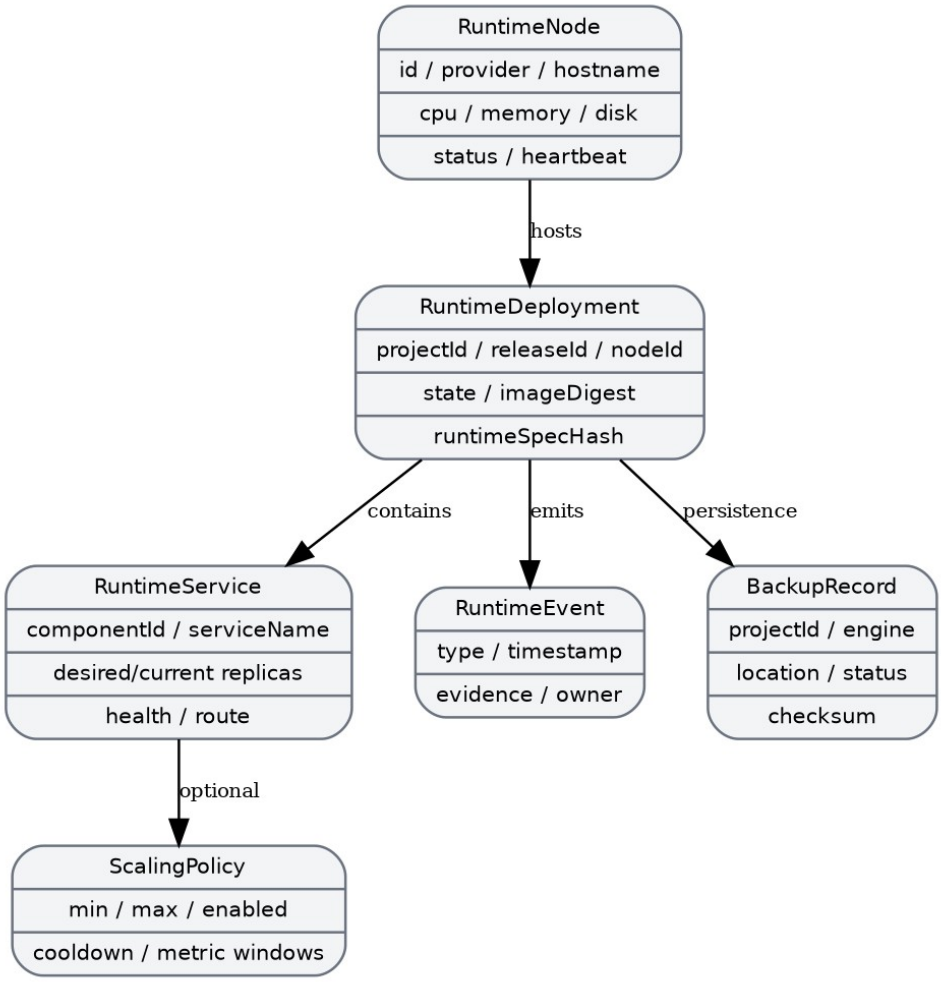


Figure 11. Proposed minimal runtime entities.

16.1 RuntimeSpec

```
RuntimeSpec {
  contractVersion
  projectId
  releaseId
  buildPlanHash

  components[] {
    componentId
    imageDigest
    command
    containerPort
    healthMode
    healthPath?
    env[]
    secretBindings[]
    resources { cpuLimit, memoryLimit, memoryReservation }
    replicaPolicy { min, max, autoscalingEligible }
  }
```

```
  relationships[] {
    fromComponent
    toComponent
    publicPrefix?
    proxyBehavior?     // PRESERVE | STRIP
    verificationPath?
  }

  managedDatabase?
  publicRoute
}
```

RuntimeSpec should be derived from BuildPlan once and hashed. Runtime Agent rejects operations whose RuntimeSpec identity does not match the operation/release.

17. UI / Dashboard Design

The self-hosted runtime should expose its value visibly. A panel demonstration is much stronger if the UI shows shared capacity, self-heal and scaling evidence rather than only 'Deployed'.

Dashboard area	Fields
Runtime node card	ONLINE/DEGRADED, CPU, memory, disk, allocatable vs reserved, running services
Project runtime card	LIVE release, replicas, CPU/memory, RPS, p95 latency, 5xx, restart count
Capacity bar	System reserve / workload reserved / currently available
Deployment timeline	Build -> preflight -> candidate -> verify -> promotion
Self-heal events	Failed task, replacement task, recovery duration
Scaling events	Old replicas -> new replicas, metric reason, capacity decision
Backup card	Last successful DB backup, age, size, restore test status
Limitations badge	Single-node: no host-level HA

17.1 Suggested runtime page wireframe

```
+-----------------------------------------------------------------+
| Runtime Node: ONLINE       CPU 43% | RAM 8.1/16 GB | Disk 38%   |
| Capacity: [system 25%][apps 49%][available 26%]                  |
+-----------------------------------------------------------------+
| Project A - LIVE      replicas 2     HEALTHY                     |
| CPU 36% | RAM 612 MB | 42 req/s | p95 118 ms | 5xx 0.1%        |
| [Open App] [Redeploy] [Rollback] [Destroy]                       |
+-----------------------------------------------------------------+
| Recent Runtime Events                                             |
| 12:04 TASK_FAILED -> 12:04 SELF_HEAL_RECOVERED (3.4 s)           |
| 12:11 SCALE 1 -> 2 (sustained load; capacity PASS)                |
+-----------------------------------------------------------------+
```

18. State and Failure Semantics

Do not create dozens of new user-facing lifecycle states. Reuse the current deployment operation model and add runtime evidence/substates. The product should still distinguish first failure ownership from later symptoms.

Condition	Recommended product meaning
Candidate cannot start	FAILED at runtime candidate stage; current LIVE unchanged

Candidate health fails	FAILED verification; no promotion
No node capacity	BLOCKED / CAPACITY_REQUIRED before candidate creation
LIVE task crashes once and recovers	LIVE remains; record SELF_HEAL event
LIVE restart loop	LIVE/DEGRADED or runtime failure evidence; alert
Node offline	PLATFORM_UNAVAILABLE; do not blame repository
Scale-out rejected by capacity	LIVE remains; emit NODE_SATURATED
Backup fails	Persistence warning; do not silently report protected backup

19. Implementation Plan

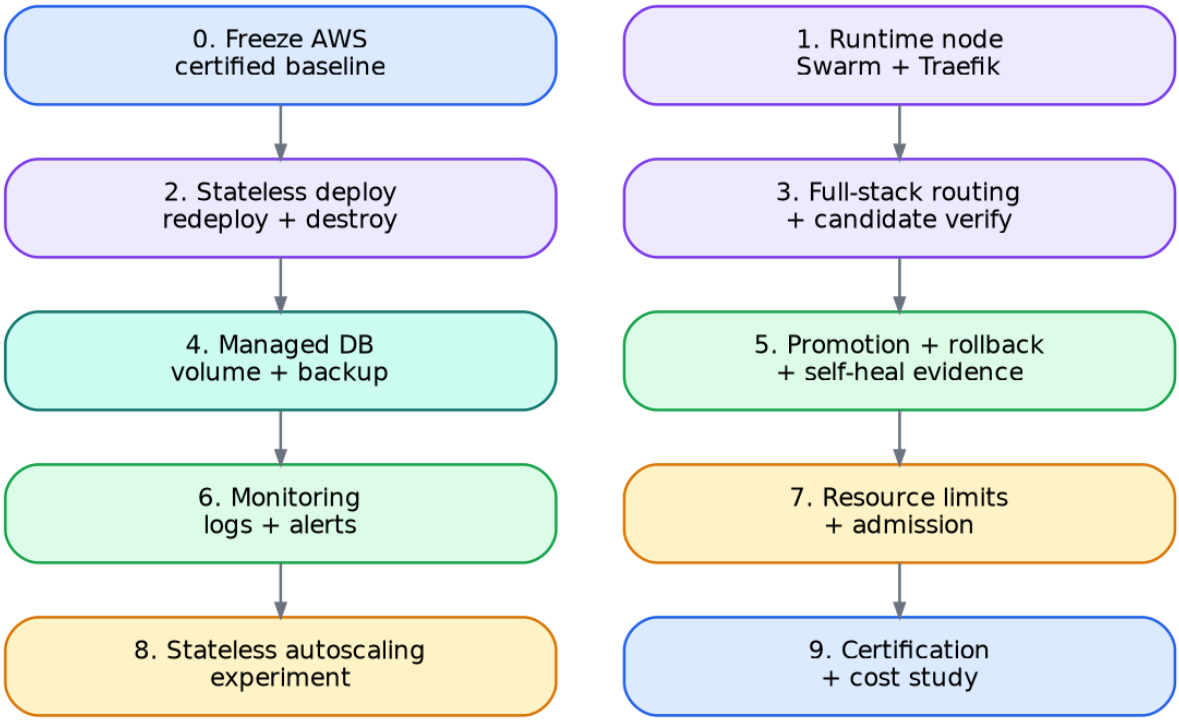


Figure 12. Recommended implementation order.

Phase 0 - Protect the baseline

- Tag/branch the current certified AWS version and keep the pinned reusable workflow/report evidence.
- Create a separate self-hosted branch.
- Do not mass-refactor detector/BuildPlan code.

Phase 1 - Bootstrap one RuntimeNode

- Provision Ubuntu/Linux VPS, Docker Engine, firewall and automatic security updates.
- Initialize single-node Swarm.
- Deploy Traefik and default wildcard DNS.
- Create RuntimeNode record and heartbeat/capacity read endpoint.

Phase 2 - Stateless backend deployment

- Introduce RuntimeSpec and Runtime Agent.
- Deploy one certified Express/FastAPI image by digest.
- Verify command, port, ENV, health, non-root and resource limits.
- Prove redeploy and destroy.

Phase 3 - Full-stack routing

- Add React/Vite frontend + backend.
- Compile BuildPlan relationship prefix/preserve/strip into runtime routing.
- Add candidate relationship verification.
- Keep API-prefix-without-GET regression test permanently.

Phase 4 - Managed databases

- Project-private PostgreSQL first, then MySQL/MongoDB.
- Named volume ownership, exact aliases and secrets.
- Logical backup + restore proof.

Phase 5 - Release reliability

- Candidate service isolation.
- Promotion and rollback.
- Docker event capture and self-heal evidence.
- Restart-loop protection.

Phase 6 - Observability

- Node exporter / Docker service metrics / Traefik metrics.
- Prometheus + Grafana dashboards.
- Alloy -> Loki logs.
- Alertmanager with small actionable rule set.

Phase 7 - Resource admission

- CPU/memory limits and reservations.
- System reserve and allocatable capacity.
- Block new deployment before node saturation.

Phase 8 - Autoscaling experiment

- Enable only on a stateless certification fixture.
- Prometheus-driven controller with hysteresis/cooldown.
- Capacity check before every scale-out.
- k6 external load test and report.

Phase 9 - Self-hosted certification

- Re-run JS/Python framework matrix through new adapter.
- Adversarial resource/security/routing tests.
- Compare immutable BuildPlan -> RuntimeSpec -> Swarm service parity.
- Document cost-per-workload and single-node limitations.

20. Test and Certification Plan

20.1 Functional corpus

- React/Vite static frontend.
- Vue/Vite static frontend.
- Express backend.
- FastAPI backend.
- Flask backend.
- Django backend.
- Node + PostgreSQL.
- Python + PostgreSQL.
- Node + MongoDB.
- Frontend + backend.
- Frontend + backend + managed database.
- Supported custom Dockerfile application.

20.2 Adversarial runtime tests

Test	Expected first boundary
Image command fails	Candidate start failure; no promotion
Wrong runtime port mutation	Runtime verification/preflight failure
Health endpoint returns 404	Candidate health failure; LIVE unchanged
API prefix only	Must NOT become health probe
Memory hog exceeds container limit	Container OOM/restart; host survives
CPU hog	Container throttled; host/platform remains responsive
Attempt privileged service	Runtime Agent rejects spec before Docker
Attempt Docker socket mount	Runtime Agent rejects spec
DB secret assigned to frontend	Contract/security gate rejects
Node capacity exhausted	Admission blocks deployment/scale-out
Kill LIVE container	Swarm self-heals and event evidence records recovery
Bad release update	No promotion / rollback
Delete DB container	Service restarts and volume data remains
Restore backup	Data integrity verification PASS

20.3 Performance / scaling tests

- Run k6 off-server.
- Establish baseline latency/RPS at one replica.
- Find CPU/memory saturation point with explicit thresholds.
- Verify 1->2 scaling changes capacity or latency behavior for a stateless workload.
- Verify scale-in only after cooldown.
- Verify scale-out refuses when node admission would be exceeded.
- Export k6 report plus Prometheus metrics for repeatability.

21. Cost and Capacity Evaluation Method

The strongest cost claim is an experimental one. Instead of comparing list prices only, define representative certified workloads, define acceptable performance thresholds, then calculate how many can coexist on the selected node.

16. Record exact VPS monthly price and renewal price.
17. Record platform overhead at idle: OS + Docker + Traefik + DeployGuard + monitoring.
18. For each fixture, measure steady-state CPU/RAM/disk.
19. Run controlled k6 load and identify an acceptable SLO envelope.
20. Pack representative workloads until the admission/saturation rule blocks additional load.
21. Repeat the test at least three times to reduce one-off noise.
22. Calculate effective infrastructure cost per admitted workload.
23. Compare with the team's measured AWS configuration using the same application and workload assumptions.

```
Do report:
  "On node X, workload profile Y admitted N applications while meeting
   threshold set Z; effective node infrastructure cost was C/N."

Do NOT report:
  "One VPS can host 20 apps" without a defined workload/SLO.
```

22. Single-Node Limitations and Honest Claims

Capability	Claim after implementation	Not allowed to claim
Container self-heal	Yes - task/process failure recovery	Physical server HA
Replica scaling	Yes - within fixed node capacity	Infinite/cloud infrastructure autoscaling

Zero-downtime candidate promotion	Yes if verified in tests	Availability through VPS outage
Database persistence	Survives service/container restart	Zero-loss database HA
Backup recovery	Yes if off-server restore is proven	No-loss recovery without off-server copy
Monitoring	Host/container/HTTP observability	Monitoring creates availability
Portability	Generic Linux RuntimeNode design	Certified on every VPS provider without testing

> **With one server, the single largest residual risk is node failure. The architecture must say this explicitly rather than hiding it behind 'self-healing'.**

23. FYP Document and Panel Positioning

The proposal explicitly states JavaScript/Python web application deployment and originally names AWS ECS Fargate as the execution target [F1]. The final system may legitimately evolve if the report clearly explains the engineering reason: cost measurements showed that per-application AWS infrastructure created a high baseline for the SME use case, so the execution plane was redesigned as pooled container infrastructure while preserving deterministic repository analysis.

23.1 Suggested final-report wording

> **Architecture evolution: The proposal initially targeted AWS ECS Fargate. During implementation and cost evaluation, the team found that the per-application cloud-resource baseline was unsuitable for the low-cost SME objective. DeployGuard therefore retained its deterministic repository-to-BuildPlan pipeline but introduced a provider-neutral self-hosted runtime that multiplexes isolated containerized web applications on shared Linux compute. AWS remains the certified reference baseline during migration.**

23.2 Panel differentiation

- DeployGuard is not merely a Docker dashboard: it derives and validates the application contract before runtime admission.
- The platform compiles repository evidence into exact ENV, port, health, routing and database contracts.
- It rejects unsupported or ambiguous repositories instead of guessing.
- It pools compute and performs resource admission so low-traffic apps do not each require a full cloud infrastructure stack.
- It demonstrates service reconciliation, self-heal, candidate promotion, rollback, monitoring and controlled scaling.
- It remains provider-portable because the RuntimeNode contract is Linux/Docker-based rather than Hostinger-specific.

24. Risks, Mitigations and Rollback Plan

Risk	Severity	Mitigation
New runtime destabilizes certified detector	High	Freeze BuildPlan boundary; no downstream re-detection
One app exhausts node	High	Mandatory cgroup limits + reservations + admission
Docker socket compromise	High	Narrow root-owned Runtime Agent; no customer socket mount
Single VPS outage	High	Explicit limitation + off-server backup + provider recovery; multi-node future
Database data loss	High	Named volume + logical off-server backup + restore tests
Autoscaling breaks stateful app	High	Stateless eligibility/opt-in; default min=max=1
Traefik routing reintroduces /api guesses	High	Compile only BuildPlan-proven prefix/preserve/strip facts
Certificate issues during repeated testing	Medium	ACME staging in development; production certificates only for final
Monitoring stack consumes too much node	Medium	Measure and include in system reserve;

capacity		retention limits
Migration takes too long	High	Keep frozen AWS baseline presentation-ready; phase-gate self-hosted work

24.1 Stop/go gates

- STOP if Phase 2 cannot deploy/redeploy/destroy one stateless certified app reliably.
- STOP expansion if runtime adapter requires changing repository evidence rules merely to function.
- Do not implement autoscaling until candidate promotion, rollback, persistence and resource isolation are stable.
- Keep AWS baseline available until the self-hosted certification report reaches the same zero-guess standard.

25. Future Multi-Node Roadmap

The RuntimeNode abstraction intentionally allows a later second node without rebuilding the repository compiler. Multi-node work is future scope and should begin only after single-node certification.

- Add worker RuntimeNodes to Swarm.
- Introduce placement policies and node labels.
- Schedule replicas across failure domains.
- Separate control plane from workload nodes.
- Introduce node-level health and rescheduling.
- Evaluate distributed persistent storage or external managed databases.
- Add node provisioning/autoscaling only when justified by measured saturation.
- At larger scale, reevaluate whether Swarm remains sufficient or whether K3s/Kubernetes becomes justified.

26. Final Recommended MVP

Capability	MVP priority
Single registered Linux RuntimeNode	MUST
Single-node Docker Swarm	MUST
Traefik host routing + HTTPS	MUST
GitHub Actions external build + immutable OCI image	MUST
BuildPlan -> RuntimeSpec projection	MUST
Runtime Agent narrow privileged API	MUST
Stateless JS/Python deployment/redeploy/destroy	MUST
Frontend + backend routing	MUST
Managed PostgreSQL; then MySQL/MongoDB	MUST for current FYP DB scope
Resource CPU/RAM limits + admission	MUST
Candidate verification + promotion/rollback	MUST
Prometheus/Grafana host/runtime metrics	MUST
Self-heal evidence	MUST
Off-server DB backup + restore proof	SHOULD
Loki centralized logs	SHOULD
Alertmanager	SHOULD
Autoscaling demonstration on stateless fixture	SHOULD / experimental
Multi-node HA	FUTURE
Node provisioning autoscale	FUTURE
Kubernetes/K3s	FUTURE

Recommended build order: deployment -> routing/TLS -> candidate/rollback -> DB/persistence -> resource isolation -> monitoring -> self-heal evidence -> autoscaling experiment. Do not start with autoscaling.

References and Research Basis

[F1] DeployGuard Project Proposal & Scope Document, 2026. Relevant sections: Scope; Rule-Based Stack Detection; System Limitations/Constraints (JavaScript/Python web applications; original AWS ECS Fargate target).

[C1] DeployGuard FYP Scope and Certification Alignment Report, 24 Aug 2026. FYP-scope certification: JS/Python, bounded frontend+backend and managed DB certified; 0 critical blind guesses, 0 critical late failures, 0 regressions.

[D1] Docker Docs - Swarm mode and services: declarative service model, scaling, desired-state reconciliation, load balancing, secure-by-default node communication, rolling updates. https://docs.docker.com/engine/swarm/

[D2] Docker Docs - Deploy services to a swarm: service desired state, replicas, resource constraints, rolling updates and rollback. https://docs.docker.com/engine/swarm/services/

[D3] Docker Docs - Resource constraints: containers have no constraints by default; memory/CPU limits protect host stability. https://docs.docker.com/engine/containers/resource_constraints/

[D4] Docker Docs - Manage sensitive data with Docker secrets: encrypted at rest/in transit in Swarm, service-scoped access, stateful service examples. https://docs.docker.com/engine/swarm/secrets/

[D5] Docker Docs - Seccomp security profiles: default profile is a moderately protective allowlist and should not generally be disabled. https://docs.docker.com/engine/security/seccomp/

[T1] Traefik Docs - Docker Swarm provider/routing: service labels and explicit load-balancer server port in Swarm mode. https://doc.traefik.io/traefik/providers/swarm/

[P1] Prometheus Docs - Monitoring Linux host metrics with Node Exporter. https://prometheus.io/docs/guides/node-exporter/

[P2] Prometheus Docs - Alertmanager: grouping, deduplication, routing, silencing and inhibition. https://prometheus.io/docs/alerting/latest/alertmanager/

[G1] Grafana Alloy Docs - Monitor Docker containers / collect metrics and logs, forward logs to Loki. https://grafana.com/docs/alloy/latest/monitor/monitor-docker-containers/

[K1] Grafana k6 Docs - Ramping VUs scenarios. https://grafana.com/docs/k6/latest/using-k6/scenarios/executors/ramping-vus/

[K2] Grafana k6 Docs - Thresholds as pass/fail performance criteria. https://grafana.com/docs/k6/latest/using-k6/thresholds/

[S1] Google Site Reliability Engineering - Monitoring Distributed Systems: four golden signals (latency, traffic, errors, saturation). https://sre.google/sre-book/monitoring-distributed-systems/

[A1] Verma et al. (2015), Large-scale cluster management at Google with Borg, EuroSys. Admission control, task packing, sharing, monitoring and fault recovery. https://research.google/pubs/large-scale-cluster-management-at-google-with-borg/

[A2] Rodriguez & Buyya (2018), Container-based Cluster Orchestration Systems: A Taxonomy and Future Directions. arXiv:1807.06193.

[A3] Buyya et al. (2018), Cost-Efficient Orchestration of Containers in Clouds: architectural framework for resource scheduling, sharing and autoscaling. arXiv:1807.03578.

[A4] Messias Filho et al. (2021), Self-Adaptive Microservice-based Systems - Landscape and Research Opportunities. arXiv:2103.08688.

[H1] Hostinger VPS pricing page, accessed 24 Aug 2026. Example KVM 4: 4 vCPU, 16 GB RAM, 200 GB NVMe; price subject to term/renewal changes. https://www.hostinger.com/vps-hosting

> **Evidence hierarchy for implementation decisions: existing DeployGuard certification and FYP scope first; official product documentation for tool capabilities second; academic/SRE literature for design principles and experimental methodology. Exact thresholds and safe workload density must be established by DeployGuard's own tests.**